

Guida Completa ai Plugin di Claude

Claude Desktop, Claude Code e Cowork

Gestione, utilizzo e applicazioni pratiche per un uso professionale avanzato

Versione 1.0 — Marzo 2026
Materiale per il corso di formazione su Claude

Indice

Indice	2
Introduzione al Documento	5
Scopo e destinatari	5
Come usare questa guida durante la formazione	5
Prerequisiti consigliati	5
Panoramica delle Ultime Funzionalità di Claude	6
1.1 Nuovi modelli e capacità	6
1.2 Claude Cowork	6
1.3 Creazione e modifica di file	6
1.4 Memoria e contesto	6
1.5 Claude in Chrome	7
1.6 Integrazioni Office	7
1.7 Tabella sinottica delle principali novità	7
Architettura dei Plugin in Claude: Concetti Fondamentali	8
2.1 Cosa sono i plugin e a cosa servono	8
2.2 Differenze tra le piattaforme	8
2.3 Plugin ufficiali vs plugin personalizzati	9
2.4 Il ruolo dei plugin nell'ecosistema Claude	9
Gestione dei Plugin in Claude Desktop	10
3.1 Struttura dei plugin: personali vs organizzativi	10
3.2 Ruoli e permessi	10
3.3 Controllo degli accessi: i 4 livelli di distribuzione	10
3.4 Come installare un plugin	11
3.5 Gestione amministrativa dei plugin	11
Upload manuale	11
Sincronizzazione GitHub	11
3.6 Personalizzazione dei plugin	11
I Plugin Ufficiali di Claude Desktop e Cowork	12
4.1 Operations	12
Skill e comandi principali	12
A chi è destinato	12
Esempio pratico	12
Modalità di attivazione	12
4.2 Design	13
4.3 Engineering	14
4.4 Data	15
4.5 Productivity	16
Funzionalità chiave	16

4.6 Enterprise Search	16
4.7 Brand Voice	17
4.8 Human Resources	17
4.9 Product Management	17
4.10 Altri plugin rilevanti	18
Modello di Estensione di Claude Code: Skills, Agents e Hooks	19
5.1 Architettura generale	19
5.2 Le Skill: definizione, struttura e funzionamento	19
Struttura di una skill	19
Trigger delle skill	19
5.3 Agenti (SubAgent): definizione, struttura e funzionamento	20
Agenti predefiniti	20
Creazione di agenti personalizzati	20
5.4 Hook: definizione, struttura e funzionamento	21
Tipi di hook	21
Principali eventi hook	21
Casi d'uso pratici	21
5.5 Differenze tra Skill, Agent e Hook	22
5.6 I comandi slash in Claude Code	22
5.7 Skill bundle predefinite in Claude Code	23
Skill e Plugin Principali di Claude Code	24
6.1 Superpowers (by Jesse Vincent / Prime Radiant)	24
Slash command principali	24
Agenti inclusi	24
Metodologia	24
Quando usarlo	24
6.2 Claude Code Setup	25
6.3 Claude MD Management	25
6.4 Code Review (Anthropic)	25
Funzionamento	25
Slash command	25
6.5 Code Simplifier (Anthropic)	26
6.6 Feature Dev (Anthropic)	26
Agenti inclusi	26
6.7 Agent SDK Dev (Anthropic)	26
Componenti	26
6.8 Firecrawl	27
6.9 Frontend Design (Anthropic)	27
6.10 Figma	28

Funzionalità principali	28
6.11 HuggingFace Skills	28
6.12 Sentry	28
6.13 SDD Workflows	29
6.14 Altri plugin rilevanti di Claude Code	37
Plugin e Cowork: Caratteristiche Specifiche	38
7.1 Il modello di estensione di Cowork	38
7.2 Differenze rispetto a Claude Desktop nella gestione dei plugin	38
7.3 Integrazioni specifiche di Cowork	38
Best Practice	39
8.1 Combinare plugin per workflow avanzati	39
8.2 Errori comuni da evitare	39
8.3 Governance in contesti enterprise	39
8.4 Sicurezza e privacy	39
Workflow: Casi d'Uso Reali	40
Workflow 1: Sviluppo software con Claude Code e Superpowers	40
Workflow 2: Gestione documentazione con Operations e Productivity	40
Workflow 3: Design collaborativo con Design e Figma	41
Workflow 4: Analisi dati con Data e Enterprise Search	41
Workflow 5: HR con Human Resources e Brand Voice	41
Riepilogo, Risorse e Riferimenti	42
10.1 Tabella sinottica di tutti i plugin	42
Plugin Claude Desktop / Cowork	42
Plugin Claude Code	43
10.2 Link alle risorse ufficiali	43
10.3 Glossario dei termini tecnici	44

Introduzione al Documento

Questa guida è stata concepita come riferimento completo per comprendere, configurare e utilizzare i plugin nelle tre principali piattaforme Claude: Claude Desktop (claude.ai), Claude Code e Claude Cowork. Il documento copre sia gli aspetti teorici che quelli pratici, con esempi concreti pensati per essere condivisi durante sessioni di formazione aziendale.

Scopo e destinatari

Il documento è destinato a professionisti, team leader, sviluppatori e amministratori IT che desiderano sfruttare al massimo le capacità di estensione offerte dall'ecosistema Claude. Che si tratti di automatizzare workflow operativi, migliorare la produttività del team di sviluppo o configurare plugin personalizzati per esigenze aziendali specifiche, questa guida fornisce le basi teoriche e gli strumenti pratici necessari.

Come usare questa guida durante la formazione

- **Sezioni 1–3:** Panoramica e concetti fondamentali — ideali per la fase introduttiva della formazione.
- **Sezioni 4–7:** Analisi dettagliata dei singoli plugin — da utilizzare come riferimento durante le esercitazioni pratiche.
- **Sezione 8:** Best practice — da condividere come linee guida operative.
- **Sezione 9:** Workflow pratici — esercizi guidati da svolgere in aula.
- **Sezione 10:** Riepilogo e risorse — materiale di consultazione post-corso.

Prerequisiti consigliati

- Un account Claude attivo (piano Pro, Max, Team o Enterprise).
- Familiarità di base con l'interfaccia di Claude Desktop (claude.ai).
- Per le sezioni su Claude Code: conoscenza basilare del terminale e di Git.
- Per le sezioni su Cowork: l'applicazione Claude Desktop installata su macOS.

Panoramica delle Ultime Funzionalità di Claude

L'ecosistema Claude ha subito un'evoluzione significativa tra il 2025 e i primi mesi del 2026, con l'introduzione di nuovi modelli, piattaforme e paradigmi di interazione. Questa sezione offre una panoramica delle novità più rilevanti per l'uso professionale.

1.1 Nuovi modelli e capacità

Claude Opus 4.6 (febbraio 2026): il modello più avanzato di Anthropic, con miglioramenti significativi nel coding, nei workflow agentic e nelle attività enterprise. Supporta una finestra di contesto da 1 milione di token (in beta) e il ragionamento esteso (extended thinking).

Claude Sonnet 4.6 (febbraio 2026): il modello bilanciato di ultima generazione, ottimizzato per la ricerca agentic con consumo ridotto di token. Combina velocità e intelligenza per le attività quotidiane.

Claude Opus 4.5 (novembre 2025): modello di frontiera con prestazioni eccezionali nel coding, computer use e workflow enterprise.

Claude Sonnet 4.5 (settembre 2025): ottimizzato per agenti reali, coding e computer use.

Claude Haiku 4.5 (ottobre 2025): il modello più rapido e conveniente, con prestazioni paragonabili a Sonnet 4 su coding e agenti.

1.2 Claude Cowork

Lanciato come research preview a gennaio 2026, Claude Cowork rappresenta una delle innovazioni più significative dell'ecosistema. Cowork è un'applicazione desktop che offre le stesse capacità di Claude Code ma con un'interfaccia grafica, pensata per professionisti non tecnici. Esegue attività in una macchina virtuale isolata sul computer dell'utente, con accesso controllato ai file locali e alla rete.

Caratteristiche principali: accesso diretto ai file locali senza upload manuali, esecuzione di task complessi come sintesi di ricerche e organizzazione documenti, coordinamento di workflow paralleli, produzione di output professionali (fogli di calcolo con formule funzionanti, presentazioni formattate), task programmati ricorrenti e on-demand.

1.3 Creazione e modifica di file

Da settembre 2025, Claude può creare e modificare direttamente file Excel, PowerPoint, documenti Word e PDF all'interno di claude.ai e dell'app desktop. Questa funzionalità, combinata con i plugin specializzati, trasforma Claude da assistente conversazionale a strumento produttivo completo.

1.4 Memoria e contesto

La funzionalità **Memory** è stata introdotta progressivamente: prima per i piani Enterprise (settembre 2025), poi Team, Max, Pro e infine per tutti gli utenti (marzo 2026). Permette a Claude di ricordare le preferenze e il contesto tra le conversazioni. La **compattazione del contesto** (novembre 2025) abilita conversazioni di lunghezza infinita tramite riassunto automatico.

1.5 Claude in Chrome

Lanciata ad agosto 2025 come estensione sperimentale del browser, Claude in Chrome permette di leggere, cliccare e navigare siti web. Successivamente espansa a tutti gli abbonati a pagamento (dicembre 2025), include supporto multi-tab, integrazione con Slack, Google Calendar, Gmail, Docs e GitHub, workflow ricorrenti e task programmati.

1.6 Integrazioni Office

Claude è ora disponibile come add-in per **PowerPoint** (febbraio 2026) e **Excel** (beta da novembre 2025), con supporto per tabelle pivot, formattazione condizionale, contesto condiviso tra applicazioni e supporto per gateway LLM aziendali (Bedrock, Vertex AI, Microsoft Foundry).

1.7 Tabella sinottica delle principali novità

Data	Novità	Piattaforma
Mar 2026	Thread persistente Cowork, visualizzazioni interattive, memoria per tutti	Desktop/Cowork
Feb 2026	Opus 4.6, Sonnet 4.6, marketplace plugin, task programmati, PowerPoint add-in	Tutte
Gen 2026	Cowork per Pro, Claude Code nei Team, HIPAA Enterprise	Cowork/Code
Dic 2025	Skills organizzative, Chrome per tutti, directory partner	Desktop/Code
Nov 2025	Opus 4.5, Excel beta, compattazione contesto, conversazioni infinite	Tutte
Set 2025	Sonnet 4.5, creazione file, memoria Enterprise/Team	Desktop
Ago 2025	Code Execution Tool, Chrome extension, artifact condivisibili	Desktop/API

Architettura dei Plugin in Claude: Concetti Fondamentali

I plugin rappresentano il meccanismo di estensione fondamentale dell'ecosistema Claude. Comprendere la loro architettura è essenziale per sfruttare appieno le capacità delle diverse piattaforme.

2.1 Cosa sono i plugin e a cosa servono

Un plugin è un pacchetto di funzionalità che estende le capacità di Claude per compiti specifici. Ogni plugin raggruppa insieme **skill** (istruzioni e competenze specializzate), **connettori** (integrazioni con servizi esterni tramite MCP), **sotto-agenti** (agenti specializzati per task specifici) e **hook** (automazioni basate su eventi). L'obiettivo è permettere a Claude di lavorare in modo più efficace in contesti professionali specifici, dalla gestione dei processi aziendali allo sviluppo software.

2.2 Differenze tra le piattaforme

Le tre piattaforme Claude adottano approcci diversi ma complementari ai plugin:

Aspetto	Claude Desktop/Cowork	Claude Code
Formato plugin	File .plugin (ZIP), marketplace organizzativi	Directory con plugin.json, marketplace GitHub
Interfaccia	GUI con browser plugin integrato	CLI con comandi /plugin
Componenti principali	Skill, connettori MCP, sotto-agenti	Skill, agenti, hook, server MCP, server LSP
Trigger	Automatico (basato su contesto) o slash command	Slash command, trigger automatico, hook su eventi
Gestione admin	Pannello organizzazione con 4 livelli di accesso	Settings file gerarchici, managed policies
Distribuzione	Upload ZIP o sync GitHub	Marketplace GitHub, --plugin-dir, /plugin install

2.3 Plugin ufficiali vs plugin personalizzati

L'ecosistema Claude distingue tra due categorie di plugin:

- **Plugin ufficiali (Anthropic & Partner):** sviluppati e verificati da Anthropic o dai partner certificati. Disponibili nel marketplace ufficiale con il badge di verifica. Coprono le esigenze più comuni: produttività, sviluppo, design, operazioni, dati.
- **Plugin personalizzati (Personal/Custom):** creati dagli utenti o dalle organizzazioni per esigenze specifiche. Possono essere distribuiti internamente tramite marketplace privati o condivisi pubblicamente. Il tool "Plugin Create" in Cowork e il plugin "plugin-dev" in Claude Code facilitano la creazione.

2.4 Il ruolo dei plugin nell'ecosistema Claude

I plugin non sono semplici add-on: rappresentano il cuore della strategia di estensibilità di Anthropic. Attraverso i plugin, le organizzazioni possono codificare i propri processi, le best practice e le integrazioni in pacchetti riutilizzabili e condivisibili, trasformando Claude in un assistente realmente personalizzato per il proprio contesto lavorativo. La portabilità tra Cowork e Claude Code (e qualsiasi applicazione basata su Agent SDK) garantisce che gli investimenti nella personalizzazione siano sostenibili nel tempo.

Gestione dei Plugin in Claude Desktop

Questa sezione illustra come installare, configurare e gestire i plugin in Claude Desktop e Cowork, sia dal punto di vista dell'utente individuale che dell'amministratore organizzativo.

3.1 Struttura dei plugin: personali vs organizzativi

Claude Desktop distingue tra due tipologie di plugin:

- **Plugin personali:** installati dall'utente e salvati localmente sulla propria macchina. Ogni utente può installarli autonomamente dal catalogo o caricando file personalizzati. Non sono visibili agli altri membri dell'organizzazione.
- **Plugin organizzativi:** distribuiti dall'amministratore dell'organizzazione tramite marketplace. Sono gestiti centralmente e possono essere resi obbligatori, opzionali o nascosti per i membri del team.

3.2 Ruoli e permessi

Il sistema di gestione dei plugin prevede due ruoli principali:

- **Amministratore (Owner/Primary Owner):** può creare marketplace, aggiungere plugin, configurare le preferenze di distribuzione e monitorare l'utilizzo. Solo i proprietari possono gestire i plugin a livello organizzativo.
- **Utente (Member):** può installare plugin resi disponibili dall'amministratore, installare plugin personali e personalizzare i plugin installati (quelli personali). Non può modificare i plugin gestiti dall'organizzazione.

3.3 Controllo degli accessi: i 4 livelli di distribuzione

Per ogni plugin nel marketplace organizzativo, l'amministratore sceglie una delle seguenti modalità di distribuzione:

Preferenza	Effetto	Esperienza utente
Installed by default	Installato automaticamente per tutti i membri	Appare nella lista dei plugin installati; l'utente può disinstallarlo
Available for install	Presente nel catalogo	L'utente lo vede nel browser e può installarlo autonomamente
Not available	Completamente nascosto	L'utente non può vederlo né accedervi
Required	Installato automaticamente senza possibilità di rimozione	Appare nella lista; non può essere disinstallato

3.4 Come installare un plugin

Procedura per l'utente:

1. Aprire Claude Desktop e passare alla scheda Cowork.
2. Cliccare su **Customize** nella barra laterale sinistra.
3. Selezionare **Browse plugins** per visualizzare le opzioni disponibili.
4. Cliccare **Install** sul plugin desiderato.
5. In alternativa, caricare un file plugin personalizzato.

Per accedere alle skill dei plugin installati, digitare "/" o cliccare il pulsante "+" durante una conversazione.

3.5 Gestione amministrativa dei plugin

Gli amministratori possono gestire i plugin organizzativi in due modi:

Upload manuale

- Caricare file ZIP individuali (max 50 MB) tramite Claude Desktop.
- Ideale per iterazione rapida e strumenti una tantum.
- Limite: 100 plugin per marketplace manuale.
- Il caricamento di un plugin con lo stesso nome sovrascrive la versione precedente.

Sincronizzazione GitHub

- Collegare un repository GitHub privato/interno per la sincronizzazione automatica.
- I repository devono essere ospitati su github.com (repository pubblici non ammessi).
- Limite: 500 plugin per marketplace sincronizzato con GitHub.
- La sincronizzazione avviene automaticamente al merge delle PR (se abilitato) o manualmente.
- I tempi di sincronizzazione possono richiedere fino a 30 minuti.

3.6 Personalizzazione dei plugin

Dopo l'installazione, ogni utente può personalizzare i plugin per adattarli al proprio flusso di lavoro. Cliccando "Customize" nell'angolo in alto a destra si apre un task Cowork dedicato alla configurazione di skill, connettori e preferenze. Questa funzionalità consente di adattare un plugin generico (es. Operations) alle specificità della propria azienda o ruolo.

Esercizio pratico: Configurazione di un plugin in Claude Desktop

1. Aprire Claude Desktop e navigare alla sezione Cowork.
2. Cliccare Customize > Browse plugins.
3. Installare il plugin "Productivity".
4. Eseguire il comando /start per inizializzare task, memoria e dashboard.
5. Aggiungere un task con linguaggio naturale: "Aggiungi alla lista: preparare la presentazione per venerdì".
6. Verificare che il task appaia nella dashboard visuale.

I Plugin Ufficiali di Claude Desktop e Cowork

Questa sezione analizza in dettaglio ciascuno dei plugin ufficiali disponibili per Claude Desktop e Cowork, sviluppati da Anthropic e dai partner verificati. Per ogni plugin vengono descritti lo scopo, le funzionalità, le modalità di utilizzo e gli esempi pratici.

4.1 Operations

Il plugin **Operations** è progettato per gestire le operazioni aziendali core, dalla documentazione dei processi alla valutazione dei fornitori, dal tracking delle richieste di modifica alla creazione di runbook operativi.

Skill e comandi principali

Slash Command	Funzione
/process-doc	Documentazione di processi aziendali: diagrammi di flusso, RACI, SOP
/vendor-review	Valutazione fornitori: analisi costi, rischi e raccomandazioni
/change-request	Creazione richieste di modifica con analisi impatto e piano di rollback
/runbook	Creazione o aggiornamento di runbook operativi per procedure ricorrenti
/status-report	Generazione report di stato con KPI, rischi e azioni
/capacity-plan	Pianificazione capacità risorse e previsioni di utilizzo
/risk-assessment	Identificazione, valutazione e mitigazione rischi operativi
/compliance-tracking	Monitoraggio requisiti di conformità e prontezza audit
/process-optimization	Analisi e miglioramento processi aziendali esistenti

A chi è destinato

Operations manager, responsabili qualità, team leader, compliance officer e chiunque gestisca processi aziendali strutturati.

Esempio pratico

Esempio: Documentazione di un processo

Comando: /process-doc Descrizione: “Documenta il processo di onboarding di un nuovo dipendente, includendo tutti i passaggi dall’accettazione dell’offerta al primo giorno di lavoro.” Claude genererà un documento strutturato con diagramma di flusso, matrice RACI e SOP dettagliata.

Modalità di attivazione

Il plugin si attiva automaticamente quando Claude rileva richieste relative a processi aziendali, oppure manualmente tramite i comandi slash. I trigger automatici includono parole chiave come “processo”, “flusso di lavoro”, “rischio”, “fornitore”, “audit”.

4.2 Design

Il plugin **Design** accelera i workflow di progettazione visiva e UX, offrendo strumenti per la critica del design, la scrittura di copy UX, audit di accessibilità e la strutturazione di piani di ricerca utente.

Slash Command	Funzione
/design-critique	Feedback strutturato su usabilità, gerarchia visiva e coerenza
/ux-copy	Scrittura e revisione di microcopy, messaggi di errore, CTA, stati vuoti
/accessibility-review	Audit WCAG 2.1 AA: contrasto colori, navigazione tastiera, screen reader
/user-research	Pianificazione e conduzione di ricerche utente
/research-synthesis	Sintesi di risultati di ricerca in temi, insight e raccomandazioni
/design-system	Audit, documentazione ed estensione del design system
/design-handoff	Generazione specifiche di handoff per sviluppatori

Destinatari: UX designer, UI designer, product designer, ricercatori UX.

Attivazione: automatica quando si discute di interfacce, design, accessibilità, o manualmente tramite slash command.

Esempio pratico

Comando: /design-critique “Revisiona questa schermata di login: il form è centrato, sfondo grigio chiaro, pulsante blu ‘Accedi’ sotto i campi email e password, link ‘Password dimenticata’ sotto il pulsante.” Claude fornirà feedback strutturato su gerarchia visiva, usabilità e coerenza.

4.3 Engineering

Il plugin **Engineering** semplifica i workflow di sviluppo software quotidiani, dall'aggiornamento per gli stand-up alla risposta agli incidenti, dalle checklist di deployment alla stesura di postmortem.

Slash Command	Funzione
/standup	Generazione aggiornamento stand-up dall'attività recente
/incident-response	Workflow di risposta agli incidenti: triage, comunicazione, postmortem
/deploy-checklist	Checklist di verifica pre-deployment
/code-review	Revisione codice per sicurezza, performance e correttezza
/debug	Sessione di debugging strutturata: riproduzione, isolamento, diagnosi, fix
/architecture	Creazione/valutazione di Architecture Decision Record (ADR)
/testing-strategy	Progettazione strategie e piani di test
/tech-debt	Identificazione, categorizzazione e prioritizzazione del debito tecnico
/system-design	Progettazione di sistemi, servizi e architetture
/documentation	Scrittura e manutenzione di documentazione tecnica

Destinatari: sviluppatori, DevOps, tech lead, engineering manager.

Attivazione: automatica in contesti di sviluppo o manualmente tramite slash command.

4.4 Data

Il plugin **Data** fornisce strumenti avanzati per l'analisi dati, la creazione di query SQL, la visualizzazione e la validazione dei risultati.

Slash Command	Funzione
/analyze	Risposta a domande sui dati, da lookup semplici ad analisi complete
/write-query	Scrittura SQL ottimizzato per tutti i dialetti principali
/create-viz	Creazione di visualizzazioni con Python (matplotlib, seaborn, plotly)
/build-dashboard	Costruzione dashboard HTML interattive con grafici, filtri e tabelle
/explore-data	Profilazione ed esplorazione di dataset
/validate-data	QA di analisi: verifica metodologia, accuratezza e bias
/statistical-analysis	Metodi statistici: statistiche descrittive, trend, outlier, test ipotesi
/sql-queries	SQL performante su tutti i dialetti (Snowflake, BigQuery, Postgres, etc.)

Destinatari: data analyst, data scientist, business analyst, data engineer.

Attivazione: automatica per richieste relative a dati, query, grafici, o manualmente.

Esempio pratico

Comando: /build-dashboard “Crea una dashboard interattiva con i dati di vendita del trimestre: KPI principali (fatturato, ordini, ticket medio), grafico trend mensile e tabella top 10 prodotti.” Claude genererà un file HTML completo con grafici interattivi e filtri.

4.5 Productivity

Il plugin **Productivity** trasforma Claude in un vero collaboratore sul posto di lavoro, con gestione task persistente, sistema di memoria a due livelli e dashboard visuale. A differenza di un semplice chatbot, Claude comprende il contesto del lavoro e agisce come un collega informato.

Slash Command	Funzione
/start	Inizializzazione del sistema: task, memoria e dashboard
/update	Aggiornamento rapido: triage task obsoleti, verifica gap di memoria
/update --comprehensive	Scansione approfondita di email, calendario e chat per azioni mancate

Funzionalità chiave

- **Gestione task:** lista in formato markdown che Claude legge, scrive e gestisce. È possibile aggiungere task in modo conversazionale.
- **Sistema di memoria:** architettura a due livelli che insegna a Claude le abbreviazioni, gli acronimi e il contesto dell'organizzazione (es. "chiedi a Marco di fare il PSR per Oracle").
- **Dashboard visuale:** interfaccia per monitorare il proprio lavoro in tempo reale.

Destinatari: tutti i professionisti che gestiscono molteplici attività e necessitano di un assistente organizzativo.

4.6 Enterprise Search

Il plugin **Enterprise Search** permette di cercare informazioni attraverso tutti gli strumenti e i documenti aziendali collegati, centralizzando la ricerca in un'unica interfaccia.

Slash Command	Funzione
/search	Ricerca trasversale su tutte le fonti connesse
/digest	Generazione di digest giornaliero/settimanale delle attività
/knowledge-synthesis	Sintesi dei risultati di ricerca da fonti multiple con attribuzione

Supporta l'integrazione con Google Workspace (Calendar, Drive, Gmail), Slack, strumenti di project management e basi di conoscenza aziendali tramite connettori MCP. Particolarmente utile per rientrare dopo un'assenza, preparare riunioni o trovare decisioni prese in passato.

4.7 Brand Voice

Il plugin **Brand Voice**, sviluppato da **Tribe AI** (partner verificato), analizza i documenti aziendali esistenti per estrarre e codificare la voce del brand in linee guida applicabili. Garantisce che tutte le comunicazioni generate da Claude mantengano un tono e uno stile coerenti con l'identità del brand.

Funzionalità principali: analisi automatica di documenti esistenti per estrarre la brand voice, generazione di linee guida stilistiche strutturate, applicazione automatica della brand voice a tutte le comunicazioni prodotte da Claude.

Destinatari: team marketing, comunicazione, content creator, brand manager.

4.8 Human Resources

Il plugin **Human Resources** supporta le operazioni HR lungo l'intero ciclo di vita del dipendente: dalla redazione di lettere di offerta alla creazione di piani di onboarding, dalla scrittura di performance review alla documentazione di politiche aziendali.

Funzionalità principali: drafting di lettere di offerta personalizzate, creazione di piani di onboarding strutturati, supporto alla stesura di performance review, generazione di documentazione HR e policy aziendali.

Destinatari: responsabili HR, people operations, recruiter, manager che gestiscono team.

4.9 Product Management

Il plugin **Product Management** (verificato da Anthropic) copre l'intero workflow del product manager: dalla scrittura di specifiche alla gestione roadmap, dalla comunicazione con gli stakeholder alla sintesi delle ricerche utente.

Slash Command	Funzione
/write-spec	Generazione PRD strutturati da problem statement o idee di feature
/roadmap-update	Pianificazione e ri-prioritizzazione roadmap (Now/Next/Later, OKR)
/stakeholder-update	Aggiornamenti personalizzati per executive, engineering o clienti
/synthesize-research	Conversione di note interviste e survey in insight strutturati
/competitive-brief	Analisi competitiva strutturata
/metrics-review	Analisi metriche di prodotto con trend e raccomandazioni
/sprint-planning	Pianificazione sprint: scoping, capacità, obiettivi

Destinatari: product manager, product owner, business analyst.

4.10 Altri plugin rilevanti

Oltre ai plugin sopra descritti, Anthropic e i partner hanno rilasciato ulteriori plugin specializzati per settori specifici:

Plugin	Descrizione	Destinatari
Financial Analysis	Ricerca di mercato, analisi competitiva, modellazione finanziaria	Analisti finanziari, CFO
Investment Banking	Revisione documenti transazionali, analisi comparabili, pitch material	Investment banker
Equity Research	Analisi trascrizioni utili, modelli finanziari, note di ricerca	Equity analyst
Private Equity	Deal sourcing, due diligence, modellazione scenari	Private equity professionals
Wealth Management	Analisi portfolio, drift, esposizione fiscale, ribilanciamento	Wealth advisor
Sales	Ricerca prospect, preparazione deal, call prep	Sales representative
Legal	Revisione documenti legali, compliance tracking	Legal counsel
Marketing	Drafting contenuti, gestione campagne	Marketing manager
Customer Support	Triage issue, drafting risposte	Support agent
Biology Research	Ricerca letteratura, design esperimenti	Ricercatori

Modello di Estensione di Claude Code: Skills, Agents e Hooks

Claude Code, lo strumento da linea di comando per lo sviluppo software agentic, adotta un modello di estensione più granulare e tecnico rispetto a Claude Desktop/Cowork. Questa sezione ne analizza l'architettura in dettaglio.

5.1 Architettura generale

Il modello di estensione di Claude Code si basa su quattro componenti principali, ciascuno con un ruolo specifico:

Componente	Definizione	Analogia
Skill	File markdown con istruzioni e competenze che Claude carica dinamicamente	Un manuale di procedure
Agente (Subagent)	Assistente AI specializzato con contesto proprio e strumenti limitati	Un collega specialista
Hook	Comando o script eseguito automaticamente in momenti specifici del ciclo di vita	Un trigger di automazione
Plugin	Pacchetto che raggruppa skill, agenti, hook e server MCP in un'unità distribuibile	Un'estensione completa

5.2 Le Skill: definizione, struttura e funzionamento

Una **skill** è un file markdown (SKILL.md) che contiene istruzioni, workflow o conoscenze di dominio che Claude può caricare quando necessario. Le skill seguono lo standard aperto Agent Skills, compatibile con diversi strumenti AI.

Struttura di una skill

Ogni skill è una directory contenente un file SKILL.md obbligatorio e file di supporto opzionali (template, esempi, script):

- **Frontmatter YAML:** metadati come name, description, disable-model-invocation, allowed-tools, context, model.
- **Contenuto markdown:** istruzioni che Claude segue quando la skill viene invocata.
- **File di supporto:** template, esempi, script che la skill può referenziare.

Trigger delle skill

- **Automatico:** Claude carica la skill quando il contesto della conversazione corrisponde alla descrizione della skill.
- **Manuale:** l'utente invoca la skill con /nome-skill.
- **Controllabile:** il campo disable-model-invocation impedisce il caricamento automatico; user-invocable: false nasconde la skill dal menu.

5.3 Agenti (SubAgent): definizione, struttura e funzionamento

Un **subagent** è un assistente AI specializzato che opera nel proprio contesto isolato, con un prompt di sistema personalizzato, accesso limitato a strumenti specifici e permessi indipendenti. I subagent permettono di preservare il contesto della conversazione principale delegando operazioni verbose o specialistiche.

Agenti predefiniti

Agente	Modello	Strumenti	Scopo
Explore	Haiku (veloce)	Solo lettura	Ricerca e analisi codebase
Plan	Ereditato	Solo lettura	Ricerca per pianificazione
general-purpose	Ereditato	Tutti	Task complessi multi-step
Bash	Ereditato	Terminale	Comandi in contesto separato

Creazione di agenti personalizzati

Gli agenti personalizzati sono definiti in file markdown con frontmatter YAML, archiviati in `.claude/agents/` (progetto) o `~/claude/agents/` (utente). Si creano tramite il comando `/agents` o manualmente. Supportano: selezione del modello, restrizione strumenti, mode di permesso, server MCP dedicati, hook specifici, memoria persistente e isolamento in worktree Git.

5.4 Hook: definizione, struttura e funzionamento

Gli **hook** sono comandi shell definiti dall'utente che vengono eseguiti in momenti specifici del ciclo di vita di Claude Code. A differenza delle skill (che forniscono istruzioni) e degli agenti (che delegano lavoro), gli hook garantiscono che determinate azioni avvengano sempre in modo deterministico, senza fare affidamento sulla decisione del modello.

Tipi di hook

Tipo	Descrizione
command	Esegue un comando shell. Il tipo più comune.
http	Invia i dati dell'evento via POST a un URL.
prompt	Valutazione LLM single-turn (decisione sì/no).
agent	Verifica multi-turn con accesso agli strumenti (subagent).

Principali eventi hook

Evento	Quando si attiva
SessionStart	All'inizio o ripresa di una sessione
PreToolUse	Prima dell'esecuzione di uno strumento (può bloccarlo)
PostToolUse	Dopo l'esecuzione riuscita di uno strumento
PermissionRequest	Quando appare un dialogo di permesso
Notification	Quando Claude invia una notifica
Stop	Quando Claude finisce di rispondere
ConfigChange	Quando un file di configurazione cambia
SubagentStart / SubagentStop	All'avvio/completamento di un subagent

Casi d'uso pratici

- Notifiche desktop quando Claude richiede input.
- Formattazione automatica del codice dopo ogni modifica (Prettier, ESLint).
- Blocco delle modifiche a file protetti (.env, package-lock.json).
- Re-iniezione del contesto dopo la compattazione della conversazione.
- Audit delle modifiche alla configurazione per compliance.
- Auto-approvazione di permessi specifici.

5.5 Differenze tra Skill, Agent e Hook

Aspetto	Skill	Agent	Hook
Natura	Istruzioni/conoscenza	Assistente autonomo	Automazione deterministica
Contesto	Condiviso (inline) o fork	Isolato (proprio contesto)	Nessun contesto LLM (tranne prompt/agent hook)
Trigger	Automatico o /comando	Delegazione da Claude o @menzione	Evento del ciclo di vita
Strumenti	Quelli della sessione (limitabili)	Configurabili per agente	Shell, HTTP, LLM
Quando usare	Aggiungere competenze/workflow	Delegare task specialistici	Automatizzare azioni deterministiche

5.6 I comandi slash in Claude Code

I comandi slash (/) sono il meccanismo principale per invocare skill e comandi in Claude Code. Esistono due categorie:

- **Comandi built-in:** /help, /compact, /init, /hooks, /agents, /permissions, /statusline, etc. Eseguono logica fissa.
- **Comandi custom (skill):** definiti nei file SKILL.md o commands/*.md. Sono prompt-based e possono orchestrare strumenti, agenti paralleli e operazioni complesse.

I comandi slash dei plugin sono sempre prefissati con il nome del plugin (es. /superpowers:brainstorming) per evitare conflitti.

5.7 Skill bundle predefinite in Claude Code

Claude Code include alcune skill predefinite disponibili in ogni sessione:

Skill	Descrizione
/batch <istruzione>	Orchestrare di modifiche su larga scala in parallelo con worktree Git isolati
/claude-api	Caricamento del riferimento API Claude per il linguaggio del progetto
/debug [descrizione]	Troubleshooting della sessione corrente leggendo il log di debug
/loop [intervallo] <prompt>	Esecuzione ripetuta di un prompt a intervalli regolari
/simplify [focus]	Revisione del codice recente per riuso, qualità ed efficienza

Esercizio pratico: Esplorare le skill in Claude Code

1. Aprire il terminale e lanciare Claude Code con: `claude` 2. Digitare `/help` per vedere tutti i comandi disponibili. 3. Digitare `/agents` per esplorare gli agenti configurati. 4. Digitare `/hooks` per visualizzare gli hook attivi. 5. Provare `/simplify` per analizzare il codice recente. 6. Installare un plugin: `/plugin install superpowers@claude-plugins-official`

Skill e Plugin Principali di Claude Code

Questa sezione analizza in dettaglio i plugin e le skill più rilevanti disponibili per Claude Code, con focus sulle funzionalità, i comandi e gli agenti inclusi.

6.1 Superpowers (by Jesse Vincent / Prime Radiant)

Superpowers è il plugin più popolare per Claude Code (oltre 230.000 installazioni), un framework di skill componibili che insegna a Claude metodologie strutturate di sviluppo software. Si basa su pratiche disciplinate come TDD (Test-Driven Development), debugging sistematico e code review integrata.

Slash command principali

Comando	Funzione
/brainstorming	Sessioni di brainstorming socratico per raffinare i requisiti prima dell'implementazione
/dispatching-parallel-agents	Coordinamento di agenti paralleli per task complessi con code review integrata
/executing-plans	Esecuzione di piani di implementazione batch con checkpoint di revisione
/finishing-a-development-branch	Completamento strutturato di un branch di sviluppo con verifica qualità
/receiving-code-review	Ricezione e gestione del feedback di code review
/requesting-code-review	Richiesta di code review strutturata con agente dedicato

Agenti inclusi

- **code-reviewer**: agente che valuta le implementazioni rispetto ai piani originali, standard di codifica e principi architetturali.

Metodologia

Superpowers impone un approccio disciplinato: il ciclo TDD “red-green-refactor” richiede che i test falliscano prima dell'implementazione. Il debugging sistematico prevede quattro fasi: investigazione causa root, analisi pattern, test ipotesi e implementazione, con revisione architetturale automatica dopo tre tentativi falliti.

Quando usarlo

Ideale per: sviluppo di feature complesse, refactoring significativi, progetti che richiedono qualità del codice elevata. Non necessario per: modifiche rapide e semplici, hotfix urgenti.

6.2 Claude Code Setup

Plugin ufficiale Anthropic per la configurazione iniziale dell'ambiente Claude Code. Guida l'utente attraverso la personalizzazione di Claude Code per un nuovo progetto, includendo la creazione del file CLAUDE.md, la configurazione dei permessi e l'impostazione delle preferenze.

Trigger: invocazione manuale durante il setup iniziale di un progetto.

6.3 Claude MD Management

Plugin dedicato alla gestione del file CLAUDE.md, il sistema di memoria basato su file di Claude Code. Fornisce strumenti per mantenere, aggiornare e organizzare le istruzioni di progetto che Claude legge all'inizio di ogni sessione. Include best practice per strutturare il file, gestire le sezioni e mantenere la coerenza tra le configurazioni.

Trigger: invocazione quando si lavora sulla configurazione del progetto o si necessita di aggiornare le istruzioni persistenti.

6.4 Code Review (Anthropic)

Plugin ufficiale Anthropic per la revisione automatizzata del codice tramite PR review con agenti multipli specializzati e filtraggio basato sul livello di confidenza per ridurre i falsi positivi. Con oltre 169.000 installazioni, è uno dei plugin più utilizzati.

Funzionamento

Il comando `/code-review` lancia un workflow di revisione PR che esegue 5 agenti Sonnet in parallelo, ciascuno specializzato in un'area: conformità CLAUDE.md, rilevamento bug, contesto storico, storia delle PR e commenti al codice. I risultati vengono aggregati e filtrati per livello di confidenza.

Slash command

- `/code-review` — Lancia il workflow completo di revisione PR.

Quando usarlo: prima di ogni merge di PR, per revisioni di sicurezza, per garantire la qualità del codice. **Quando evitarlo:** per modifiche banali (typo, commenti).

6.5 Code Simplifier (Anthropic)

Plugin ufficiale (oltre 140.000 installazioni) che semplifica e raffina il codice recentemente modificato preservandone la funzionalità. Lancia tre agenti di revisione in parallelo che analizzano il codice per riuso, qualità ed efficienza, poi aggrega i risultati e applica le correzioni.

Comando: `/simplify [focus]` — Revisione del codice con focus opzionale (es. “focus on memory efficiency”).

Trigger: manuale tramite slash command.

6.6 Feature Dev (Anthropic)

Plugin ufficiale (oltre 131.000 installazioni) che implementa un workflow strutturato di sviluppo feature in 7 fasi, dalla comprensione dei requisiti alla revisione finale.

Agenti inclusi

- **code-explorer:** esplorazione e analisi della codebase esistente.
- **code-architect:** progettazione architetturale della feature.
- **code-reviewer:** revisione qualità dell'implementazione.

Comando: `/feature-dev` — Lancia il workflow guidato di sviluppo feature.

Quando usarlo: sviluppo di nuove feature significative che richiedono pianificazione. **Quando evitarlo:** bug fix semplici, modifiche minori.

6.7 Agent SDK Dev (Anthropic)

Kit di sviluppo per la creazione e verifica di applicazioni basate su Claude Agent SDK in Python e TypeScript. Semplifica l'intero ciclo di vita: dallo scaffolding iniziale alla verifica rispetto alle best practice.

Componenti

- `/new-sdk-app` — Setup interattivo per nuovi progetti Agent SDK (scelta linguaggio, configurazione ambiente, installazione SDK).
- **agent-sdk-verifier-py** e **agent-sdk-verifier-ts:** agenti che verificano le applicazioni SDK rispetto alle best practice.

6.8 Firecrawl

Plugin che integra capacità avanzate di web scraping, crawling e ricerca direttamente in Claude Code, con rendering JavaScript automatico, gestione anti-bot e rotazione proxy integrati.

Comando	Funzione
<code>/firecrawl:scrape</code>	Estrazione di una singola pagina web come markdown pulito
<code>/firecrawl:crawl</code>	Crawling e estrazione contenuti da un intero sito web
<code>/firecrawl:search</code>	Ricerca web con risultati scraping
<code>/firecrawl:map</code>	Scoperta di tutti gli URL di un sito
<code>/firecrawl:agent</code>	Agente AI autonomo per trovare e estrarre dati specifici

Quando usarlo: ricerca tecnica, estrazione dati da documentazione web, analisi competitiva basata su contenuti web.

6.9 Frontend Design (Anthropic)

Il plugin più installato in assoluto (oltre 371.000 installazioni), progettato per aiutare Claude a generare interfacce frontend di qualità professionale, evitando l'estetica generica tipica del codice generato dall'AI.

La skill frontend-design si attiva **automaticamente** quando Claude lavora su frontend, fornendo linee guida su scelte di design audaci, tipografia, animazioni e dettagli visivi. Non richiede invocazione manuale.

Quando usarlo: qualsiasi lavoro frontend dove la qualità visiva conta. Il plugin è particolarmente efficace combinato con il plugin Figma per tradurre design in codice.

6.10 Figma

Plugin che collega Claude Code direttamente ai file di design Figma, consentendo di accedere al contesto del design, estrarre informazioni sui componenti, recuperare token di design e tradurre i design in codice production-ready con fedeltà pixel-perfect.

Funzionalità principali

- Estrazione di dati strutturati di design (layout, tipografia, colori) dai frame Figma.
- Recupero di variabili e token di design.
- Mapping dei componenti Figma alla codebase esistente tramite Code Connect.
- Cattura di riferimenti visivi per la validazione.

Trigger: configurazione tramite server MCP Figma; si attiva quando si lavora su implementazioni di design.

6.11 HuggingFace Skills

Plugin che fornisce integrazione con la piattaforma HuggingFace per costruire, addestrare, valutare e utilizzare modelli AI open source, dataset e Spaces. Permette di cercare modelli, consultare la documentazione HuggingFace, esplorare paper di ricerca e gestire repository direttamente da Claude Code.

Funzionalità: ricerca modelli e dataset, consultazione documentazione, ricerca paper, gestione repository, generazione immagini con modelli hosted.

Trigger: automatico quando si lavora con modelli ML o si menziona HuggingFace.

6.12 Sentry

Plugin per il monitoraggio errori che permette di accedere ai report di errore, analizzare stack trace, cercare issue e fare debug di errori di produzione direttamente nell'IDE o nel terminale.

Funzionalità principali: accesso ai report di errore di Sentry, analisi delle stack trace, ricerca e filtraggio delle issue, debugging guidato degli errori di produzione.

Trigger: configurazione tramite server MCP Sentry; si attiva quando si lavora su bug fixing o monitoraggio errori.

6.13 SDD Workflows

Prima che venissero introdotti i plugin, la metodologia SDD era basata sul seguente workflow: "scrivo un prompt strutturato → planning mode → genero un piano di sviluppo → eseguo il piano di sviluppo".

Questo è appunto la natura del **workflow SDD (Spec-Driven Development)**, che usa il **planning mode nativo di Claude Code**.

I plugin semplificano l'adozione della metodologia SDD, in quanto implementano nativamente un flusso di lavoro guidato.

In questo ambito assumono particolare rilevanza i plugin `/superpowers` e `/feature-dev`:

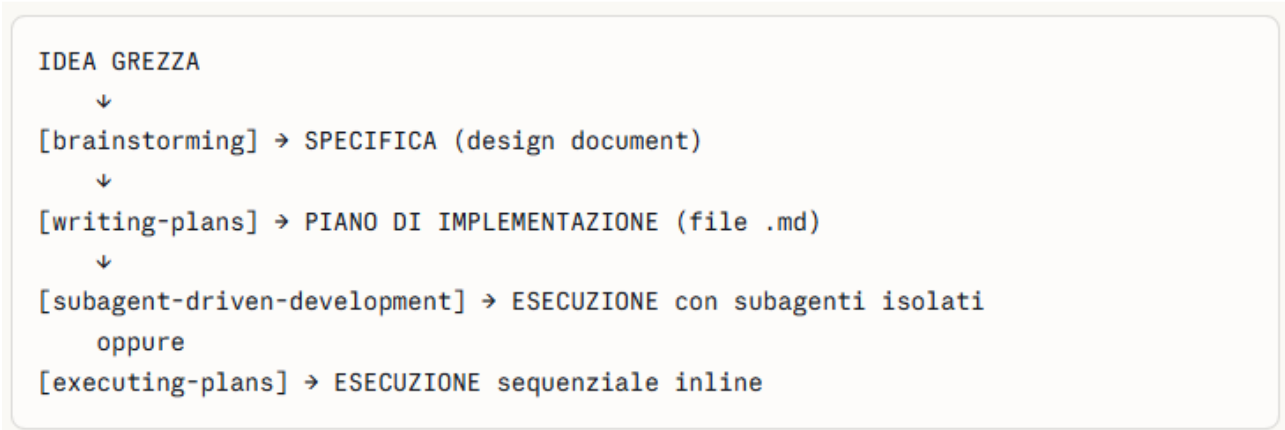
	Superpowers	/feature-dev
Origine	Plugin di terze parti (Jesse Vincent / obra)	Plugin ufficiale Anthropic
Scope	Metodologia completa SDLC	Workflow specifico per feature
Agenti	Subagent per task + code reviewer	<code>code-explorer</code> , <code>code-architect</code> , <code>code-reviewer</code>
Filosofia	Disciplina forzata dall'inizio	Guided workflow più leggero

Superpowers: architettura e workflow

Superpowers è un framework completo di skills che implementa metodologie strutturate di sviluppo software — TDD, debugging sistematico, brainstorming, subagent-driven development con code review integrata.

Le tre skills principali e la loro sequenza

Il workflow di Superpowers è **rigorosamente sequenziale** e si compone di fasi obbligate:



Fase 1: brainstorming — dalla vaga idea alla specifica

Il sistema non salta direttamente alla scrittura di codice. Invece, si ferma e chiede cosa stai *veramente* cercando di fare. Una volta estratta una spec dalla conversazione, te la mostra in chunk abbastanza brevi da leggere e digerire effettivamente.

Questo NON è un semplice brainstorming creativo: è un **dialogo socratico strutturato** che produce un artefatto concreto. Claude fa domande mirate su:

- Requisiti funzionali e casi limite (edge cases)
- Architettura e trade-off tecnici
- Cosa è dentro scope e cosa è fuori (YAGNI)

Output: un documento spec salvato su disco (es. docs/superpowers/specs/YYYY-MM-DD-feature-name.md)

qui sta la differenza fondamentale rispetto al planning mode tradizionale. Il planning mode tradizionale produce un *piano di implementazione*. Il brainstorming di Superpowers produce una *specifica funzionale/architetturale* — un livello di astrazione superiore, più vicino a un documento di design che a una to-do list di coding.

Fase 2: writing-plans — dalla specifica al piano eseguibile

Il piano generato include un header esplicito per agenti agentic: è richiesto l'uso di superpowers:subagent-driven-development (raccomandato) oppure superpowers:executing-plans per implementare il piano task per task.

Il piano è abbastanza chiaro da essere seguito da un "junior engineer entusiasta ma senza buon gusto, senza senso del giudizio, senza contesto di progetto, e con avversione per i test". Enfatizza TDD red/green, YAGNI e DRY.

Ogni task nel piano:

- Ha boundaries chiari e interfacce definite
- Specifica esattamente i file da creare/modificare
- Include criteri di verifica (i test devono fallire prima dell'implementazione)
- È atomico: producibile da un subagente fresco senza contesto esterno

Fase 3a: subagent-driven-development — l'approccio raccomandato

Esegui il piano dispatching un subagente fresco per ogni task, con review in due stadi dopo ciascuno: prima spec compliance review, poi code quality review. Il principio core è: subagente fresco per task + review in due stadi = alta qualità, iterazione veloce.

```
[Orchestratore legge piano]
  → Dispatcha subagente implementatore (contesto isolato)
    → Implementa il task con TDD
    → Self-review
    → Commit
  → Dispatcha spec compliance reviewer
    → Verifica che implementation = spec (né meno né più)
  → Solo se compliance OK → Dispatcha code quality reviewer
    → Valuta qualità del codice
  → Passa al task successivo
```

Questa è la parte più sorprendente: il workflow presenta due opzioni di esecuzione. Subagenti nella sessione corrente (buono per task semplici che condividono contesto) oppure orchestrazione di agenti isolati — aprire una sessione Claude Code separata nella stessa cartella con un nuovo agente che esegue il piano, mentre l'agente originale funge da project manager. I sub-agenti aiutano a fare due cose che altrimenti non si potrebbero fare: gestire la context window e fare task in parallelo. Se si vuole output di alta qualità, non si vuole lavorare con una context window piena. Claude Code gestisce automaticamente la context window e man mano che ci si avvicina al limite avvisa con un messaggio di compaction.

Il meccanismo concreto di isolamento:

1. **L'orchestratore** mantiene un contesto piccolo e focalizzato sulla coordinazione
2. **Ogni subagente implementatore** riceve esattamente e solo ciò che serve per *quel* task: il testo del task, il contesto architetturale minimo, le istruzioni TDD
3. Il subagente non "inquina" il contesto dell'orchestratore con dettagli implementativi
4. Questo consente sessioni di sviluppo **di ore** senza degradazione della qualità

Fase 3b: executing-plans — l'alternativa sequenziale

L'opzione executing-plans è riservata agli harness senza supporto per subagenti. Nei nuovi rilasci, il sistema indica esplicitamente che Superpowers funziona meglio su piattaforme con supporto subagenti.

In sostanza: executing-plans = fallback per ambienti che non supportano subagenti (come alcune versioni di Codex o tool alternativi). Su Claude Code standard, il percorso raccomandato è subagent-driven-development.

/feature-dev: il workflow Anthropic ufficiale

Il comando /feature-dev offre un guided feature development workflow con tre agenti specializzati: code-explorer per l'analisi del codebase, code-architect per il design architetturale, e code-reviewer per la validazione della qualità.

Rispetto a Superpowers:

- **Più leggero:** non forza brainstorming preventivo né TDD
- **Più integrato:** è built-in nel plugin ufficiale Anthropic
- **Più focalizzato:** pensato per *aggiungere feature* a codebase esistenti, non per progetti from-scratch
- **Meno prescrittivo:** non impone una metodologia completa come Superpowers

E un workflow autocontenuto in 7 fasi per implementare una singola feature:

Fase	Cosa fa
1. Discovery	Capisce cosa costruire, fa domande
2. Codebase Exploration	Lancia 2-3 agenti <code>code-explorer</code> in parallelo per capire il codice esistente
3. Clarifying Questions	Identifica ambiguità, edge case, chiede conferme
4. Architecture Design	Lancia 2-3 agenti <code>code-architect</code> con approcci diversi, propone alternative
5. Implementation	Implementa (solo dopo approvazione utente)
6. Quality Review	Lancia 3 agenti <code>code-reviewer</code> in parallelo
7. Summary	Documenta il lavoro fatto

Differenze chiave con il percorso /superpowers

	/feature-dev	/superpowers (brainstorm → plan → execute)
Scope	Una feature alla volta	Progetto intero o sub-progetto
Spec	No spec formale, si parte dal codice	Spec scritta e committata
Piano	Implicito nell'architettura	Piano esplicito con step e checkpoint
Quando usarlo	Hai già un codebase e aggiungi una feature	Parti da zero o ridisegni qualcosa

Il confronto con l'approccio tradizionale (Planning Mode)

Dimensione	Planning Mode tradizionale	Superpowers
Input iniziale	Descrizione testuale della feature	Dialogo socratico → spec document
Output pianificazione	Piano di implementazione inline	File .md strutturato su disco
Esecuzione	Sequenziale nel context corrente	Subagente fresco per ogni task
Review	Nessuna review strutturata	2-stage review post-task
Persistenza	Nella conversazione (volatile)	Su file system (permanente)
Context drift	Alta probabilità in sessioni lunghe	Eliminata per design
TDD	Opzionale	Obbligatorio per ogni subagente

Superpowers forza un processo ingegneristico disciplinato attraverso skills obbligatori che il modello non può aggirare. Gli skill forniscono quella struttura essenziale e non negoziabile. Convogliano l'intelligenza grezza degli agenti, assicurando che ogni missione sia eseguita con precisione invece di concludersi in un caos.

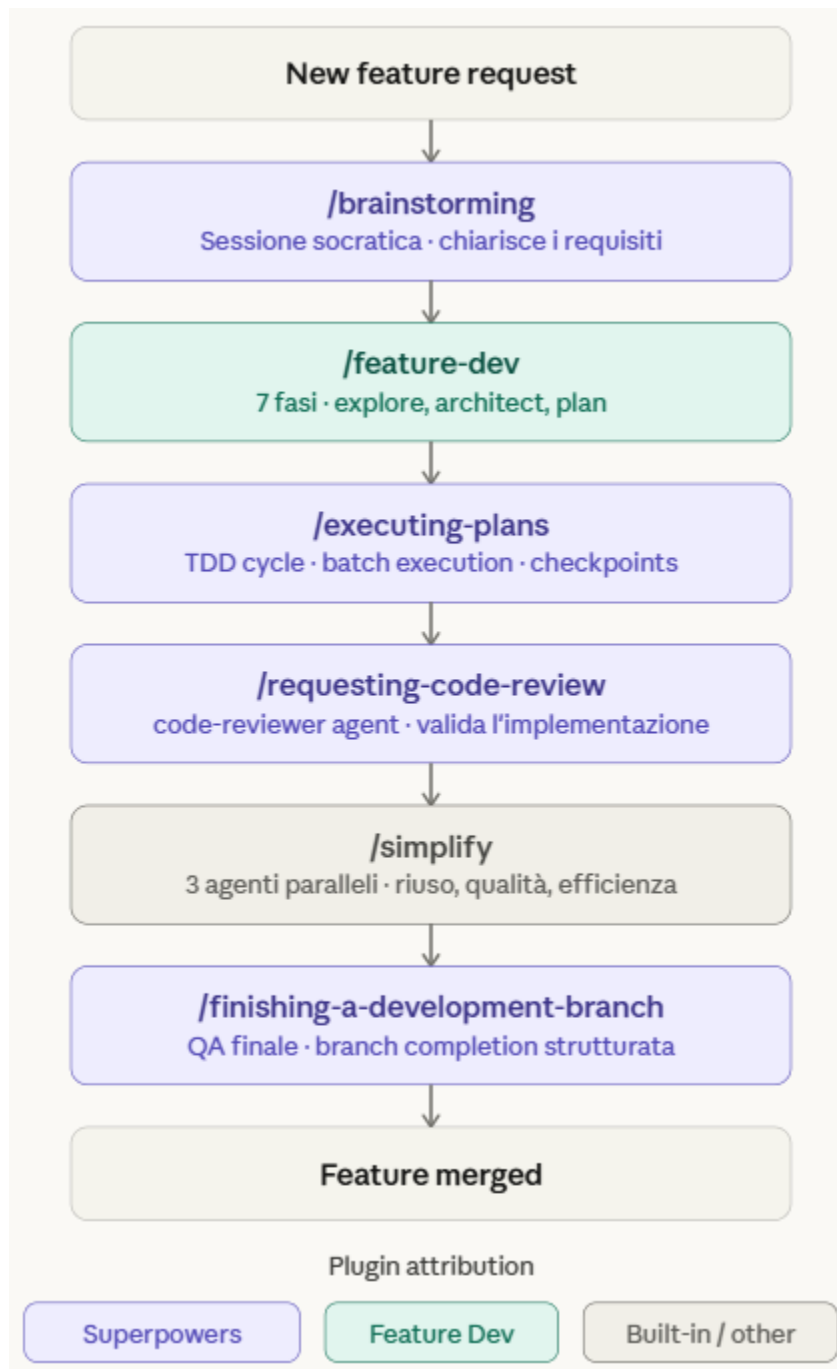
Evoluzione della metodologia spec-driven:

1. **Livello 1 (base):** Planning mode → genera piano → esegui (un contesto, sequenziale)
2. **Livello 2 (intermedio):** Spec scritta manualmente → implement con subagenti (separazione spec/esecuzione)
3. **Livello 3 (avanzato):** Superpowers → brainstorming genera spec → writing-plans genera piano → subagent-driven-development esegue con review (pipeline completamente orchestrata)

Integrazione tra /superpowers e /feature-dev

In questo paragrafo verrà illustrato un approccio per integrare l'uso dei due plugin descritti.

La logica di fondo è semplice: **Superpowers** è il layer di *metodologia* (come sviluppare bene: TDD, debugging sistematico, review discipline), mentre **Feature Dev** è il layer di *orchestrazione del workflow* (quali fasi seguire in ordine). Non sono alternativi — sono complementari per natura.



Fase 1 — Upstream (pre-codice)

`/brainstorming` è il vero punto di partenza. Prima di aprire la codebase, Superpowers conduce una **sessione socratica**: Claude pone domande mirate per estrarre requisiti funzionali, non-funzionali e vincoli tecnici. L'output è un documento di requisiti chiaro e condiviso, non ancora codice.

Fase 2 — Planning (struttura)

`/feature-dev` riceve in input il documento prodotto da `/brainstorming` e attiva il workflow:

1. Comprensione requisiti
2. Esplorazione codebase (agente code-explorer)
3. Progettazione architetturale (agente code-architect)
4. Breakdown in task 5–7. Pianificazione, revisione del piano, approvazione

Il punto critico è che **Feature Dev analizza il codice esistente** prima di pianificare qualsiasi modifica — evita soluzioni in conflitto con l'architettura già in piedi.

Fase 3 — Execution (implementazione)

`/executing-plans` prende il piano di Feature Dev e lo esegue con la disciplina di Superpowers: ciclo **TDD red-green-refactor**, checkpoint espliciti dopo ogni blocco di modifiche, e — per feature complesse — `/dispatching-parallel-agents` per parallelizzare il lavoro su worktree Git isolati.

Nota: `/dispatching-parallel-agents` non è nel flow principale perché è opzionale, ma è il comando da usare quando il piano ha task indipendenti che possono girare in parallelo.

Fase 4 — Quality & Close

La sequenza finale è sempre la stessa:

- **`/requesting-code-review`** → lancia l'agente code-reviewer che valuta l'implementazione rispetto al piano originale e agli standard architetturali
- **`/simplify`** → 3 agenti paralleli ottimizzano il codice per riuso, qualità ed efficienza (Code Simplifier)
- **`/finishing-a-development-branch`** → Superpowers chiude il branch con verifica qualità strutturata (message di commit, checklist, PR summary)

Il workflow funziona perché i due plugin si passano un **artefatto intermedio** ben definito:

```
/brainstorming → produce: documento requisiti
↓
/feature-dev → consuma: documento requisiti
               → produce: piano implementazione dettagliato
↓
/executing-plans → consuma: piano implementazione
```

Se salti `/brainstorming` e vai direttamente a `/feature-dev`, Feature Dev parte ugualmente — ma rischia di pianificare la feature sbagliata. Se salti `/feature-dev` e vai direttamente a `/executing-plans`, Superpowers esegue — ma senza un piano architetturale solido.

La combinazione dei due elimina il gap più comune nello sviluppo AI-assisted: **codice tecnicamente corretto ma funzionalmente sbagliato**.

`/brainstorming` → `/feature-dev` non è un trigger automatico.

I due comandi sono slash command separati e indipendenti — li invochi tu manualmente in sequenza. Quello che succede è:

- `/brainstorming` termina producendo un documento di requisiti (tipicamente salvato come file nella sessione)
- A quel punto **sei tu** che invochi `/feature-dev`, passandogli quel documento come contesto

Il flusso è sequenziale ma **human-in-the-loop**: tu decidi quando il brainstorming è abbastanza maturo per passare alla fase successiva. Questo è intenzionale — evita che Claude pianifichi su requisiti ancora incompleti.

6.14 Altri plugin rilevanti di Claude Code

Plugin	Descrizione	Installazioni
PR Review Toolkit	Suite di agenti per revisione PR: commenti, test, errori, tipi, qualità, semplificazione	Significative
Ralph Loop	Loop iterativi interattivi per sviluppo: Claude lavora su un task ripetutamente fino al completamento	110.000+
Plugin Dev	Toolkit completo per lo sviluppo di plugin con 7 skill esperte e creazione assistita da AI	Significative
Security Guidance	Hook che monitora 9 pattern di sicurezza: command injection, XSS, eval, HTML pericoloso	25.000+
Hookify	Creazione facile di hook personalizzati analizzando pattern di conversazione	Significative
Commit Commands	Automazione workflow Git: /commit, /commit-push-pr, /clean_gone	Significative
Skill Creator	Creazione, modifica e misurazione delle performance delle skill	Significative
Playwright	Automazione browser e testing end-to-end tramite server MCP Microsoft	118.000+
GitHub	Server MCP GitHub ufficiale per gestione repository, issue e PR	141.000+
Context7	Server MCP per lookup documentazione live	189.000+
MCP Server Dev	Sviluppo di server MCP personalizzati	Significative
Plugin LSP (TypeScript, Python, Rust, Go, etc.)	Integrazione Language Server Protocol per code intelligence in tempo reale	Varie

Plugin e Cowork: Caratteristiche Specifiche

Claude Cowork, pur condividendo il formato dei plugin con Claude Code e l'Agent SDK, presenta caratteristiche specifiche pensate per un pubblico non tecnico e per l'uso in contesti enterprise.

7.1 Il modello di estensione di Cowork

Cowork adotta un approccio "plug and play": i plugin si installano con un clic dall'interfaccia grafica, senza necessità di configurazione tecnica. Ogni plugin include skill, connettori e sotto-agenti già pre-configurati. L'utente interagisce con i plugin tramite linguaggio naturale o comandi slash, e può personalizzarli attraverso una conversazione guidata con Claude.

7.2 Differenze rispetto a Claude Desktop nella gestione dei plugin

Aspetto	Claude Desktop (claude.ai)	Claude Cowork
Esecuzione	Nel cloud Anthropic	In una VM locale isolata sul computer dell'utente
Accesso file	Solo file caricati manualmente	Accesso diretto ai file locali
Plugin marketplace	Non disponibile direttamente	Integrato con browser plugin e Customize
Creazione plugin	Non supportata nativamente	Plugin Create integrato
Task programmati	Non disponibili	Task ricorrenti e on-demand
Connettori MCP	Server MCP locali (configurazione manuale)	Connettori integrati con setup guidato
Destinatari primari	Utenti generali, conversazioni	Knowledge worker, task complessi, automazione

7.3 Integrazioni specifiche di Cowork

Cowork supporta oltre 20 connettori pre-configurati che permettono a Claude di interagire con servizi esterni:

- **Google Workspace:** Calendar, Drive, Gmail per la gestione della comunicazione e dei documenti.
- **Strumenti di vendita:** Apollo, Clay, Outreach, Similarweb per ricerca prospect e preparazione deal.
- **Servizi legali e firma:** DocuSign, LegalZoom per la gestione documenti legali.
- **Dati finanziari:** MSCI, FactSet per analisi finanziarie e di mercato.
- **Pubblicazione:** WordPress per la gestione contenuti.
- **AI specializzata:** Harvey per servizi legali AI.

Best Practice

8.1 Combinare plugin per workflow avanzati

La vera potenza dei plugin emerge quando vengono combinati. Alcuni esempi:

- **Sviluppo feature:** Superpowers (metodologia) + Feature Dev (workflow) + Code Review (qualità) + Frontend Design (UI).
- **Analisi dati enterprise:** Data (query e visualizzazione) + Enterprise Search (recupero dati) + Productivity (tracking risultati).
- **Comunicazione aziendale:** Brand Voice (tono) + Product Management (stakeholder update) + Operations (status report).

8.2 Errori comuni da evitare

- **Sovraccaricare Claude con troppi plugin:** ogni plugin aggiunge contesto; troppi plugin possono superare il budget di contesto disponibile e degradare le prestazioni.
- **Non personalizzare i plugin:** un plugin generico è utile, ma uno personalizzato per il proprio contesto è molto più efficace.
- **Ignorare la governance:** in contesti enterprise, non definire chi può installare cosa può portare a proliferazione incontrollata e rischi di sicurezza.
- **Confondere skill e hook:** le skill forniscono istruzioni (Claude può decidere); gli hook eseguono azioni deterministiche (sempre eseguiti).
- **Non aggiornare i plugin:** i plugin evolvono; mantenere versioni obsolete può causare incompatibilità o mancanza di funzionalità.

8.3 Governance in contesti enterprise

- **Definire una policy per i plugin:** stabilire chi può creare, approvare e distribuire plugin nell'organizzazione.
- **Utilizzare marketplace privati:** centralizzare la distribuzione tramite repository GitHub privati sincronizzati.
- **Revisionare periodicamente:** pianificare revisioni trimestrali dei plugin attivi per eliminare quelli obsoleti e aggiornare quelli in uso.

8.4 Sicurezza e privacy

- **Esecuzione isolata:** Cowork esegue i plugin in una VM isolata sul computer dell'utente, limitando l'accesso ai file e alla rete.
- **Plugin da fonti affidabili:** installare solo plugin dal marketplace ufficiale, da partner verificati o da repository interni controllati.
- **Revisione del codice dei plugin personalizzati:** prima di distribuire un plugin personalizzato, revisionarne il codice (specialmente hook e script).
- **Permessi minimi:** configurare gli agenti dei plugin con i permessi minimi necessari (principio del privilegio minimo).
- **Attenzione ai connettori:** ogni connettore MCP è un punto di integrazione con servizi esterni; valutare attentamente quali dati vengono condivisi.

Workflow: Casi d'Uso Reali

Questa sezione presenta cinque workflow completi che combinano diversi plugin per risolvere scenari professionali reali. Ogni workflow è pensato per essere riprodotto durante le sessioni di formazione.

Workflow 1: Sviluppo software con Claude Code e Superpowers

Scenario: sviluppo di una nuova feature di autenticazione in un progetto esistente.

1. **Fase 1 — Brainstorming:** `/brainstorming` — Sessione socratica per definire i requisiti della feature di autenticazione. Claude pone domande per chiarire requisiti funzionali, non funzionali e vincoli tecnici.
2. **Fase 2 — Esplorazione:** Claude utilizza automaticamente l'agente Explore per analizzare la codebase esistente e comprendere l'architettura corrente.
3. **Fase 3 — Pianificazione:** `/feature-dev` — Il workflow strutturato in 7 fasi guida dalla comprensione dei requisiti alla progettazione architetturale.
4. **Fase 4 — Implementazione:** `/executing-plans` — Esecuzione del piano con checkpoint di revisione. Se necessario, `/dispatching-parallel-agents` per parallelizzare il lavoro.
5. **Fase 5 — Revisione:** `/requesting-code-review` — L'agente code-reviewer valuta l'implementazione. Poi `/simplify` per ottimizzare il codice.
6. **Fase 6 — Finalizzazione:** `/finishing-a-development-branch` — Completamento strutturato con verifica qualità finale.
- 7.

Workflow 2: Gestione documentazione con Operations e Productivity

Scenario: documentazione dei processi aziendali di un team in crescita.

1. **Setup:** installare Operations e Productivity in Cowork. Eseguire `/start` per inizializzare il sistema Productivity.
2. **Inventario processi:** chiedere a Claude di creare una lista dei processi da documentare. Productivity li traccia come task.
3. **Documentazione:** per ogni processo, `/process-doc` genera documentazione strutturata con diagrammi di flusso e matrici RACI.
4. **Valutazione fornitori:** `/vendor-review` per confrontare fornitori di strumenti necessari.
5. **Report di stato:** `/status-report` settimanale con KPI sull'avanzamento della documentazione.

Workflow 3: Design collaborativo con Design e Figma

Scenario: traduzione di un design Figma in codice frontend di qualità.

1. **Analisi design:** con il plugin Figma configurato, Claude legge i frame dal file Figma, estraendo layout, tipografia, colori e componenti.
2. **Revisione design:** /design-critique per ottenere feedback strutturato sull'usabilità e la coerenza del design.
3. **Accessibilità:** /accessibility-review per verificare la conformità WCAG 2.1 AA.
4. **Implementazione:** con Frontend Design attivo, Claude genera codice frontend di qualità professionale fedele al design originale.
5. **Handoff:** /design-handoff per generare le specifiche complete per il team di sviluppo.

Workflow 4: Analisi dati con Data e Enterprise Search

Scenario: preparazione di un report trimestrale sulle vendite.

1. **Raccolta dati:** /search per trovare tutti i documenti e le discussioni rilevanti sulle vendite del trimestre attraverso gli strumenti aziendali collegati.
2. **Query dati:** /write-query per generare le query SQL necessarie ad estrarre i dati dalle fonti dati aziendali.
3. **Esplorazione:** /explore-data per profilare i dataset e identificare pattern, anomalie e qualità dei dati.
4. **Analisi:** /analyze per l'analisi approfondita dei trend e delle performance.
5. **Visualizzazione:** /build-dashboard per creare una dashboard interattiva con KPI, grafici trend e tabelle di dettaglio.
6. **Validazione:** /validate-data per verificare metodologia, accuratezza e potenziali bias prima della presentazione.

Workflow 5: HR con Human Resources e Brand Voice

Scenario: onboarding di 10 nuovi dipendenti con comunicazione coerente.

1. **Setup Brand Voice:** installare il plugin Brand Voice e fornirgli i documenti aziendali esistenti (presentazioni, comunicazioni interne, manuale del brand). Claude analizza e codifica la voce del brand.
2. **Lettere di offerta:** utilizzare il plugin HR per generare lettere di offerta personalizzate per ciascun candidato, nel tono del brand aziendale.
3. **Piani di onboarding:** creare piani di onboarding strutturati e personalizzati per ruolo.
4. **Comunicazione interna:** generare l'annuncio interno dei nuovi ingressi, automaticamente nel tono del brand.
5. **Tracking:** utilizzare Productivity per tracciare lo stato di ogni onboarding e impostare task ricorrenti per i check-in a 30, 60 e 90 giorni.

Riepilogo, Risorse e Riferimenti

10.1 Tabella sinottica di tutti i plugin

Plugin Claude Desktop / Cowork

Plugin	Categoria	Sviluppatore	Trigger
Operations	Workflow e processi	Anthropic	Automatico + slash command
Design	Design e UX	Anthropic	Automatico + slash command
Engineering	Sviluppo software	Anthropic	Automatico + slash command
Data	Analisi dati	Anthropic	Automatico + slash command
Productivity	Produttività personale	Anthropic	Slash command (/start, /update)
Enterprise Search	Ricerca enterprise	Anthropic	Automatico + slash command
Brand Voice	Comunicazione	Tribe AI (Partner)	Automatico
Human Resources	Risorse umane	Anthropic	Automatico + slash command
Product Management	Gestione prodotto	Anthropic	Slash command
Financial Analysis	Finanza	Anthropic	Automatico + slash command
Sales	Vendite	Anthropic	Automatico + slash command
Legal	Legale	Anthropic	Automatico + slash command

Plugin Claude Code

Plugin	Categoria	Sviluppatore	Installazioni
Frontend Design	UI/Frontend	Anthropic	371.000+
Superpowers	Metodologia sviluppo	Jesse Vincent	233.000+
Context7	Documentazione live	Upstash	189.000+
Code Review	Qualità codice	Anthropic	169.000+
GitHub	Gestione repository	GitHub	141.000+
Code Simplifier	Semplificazione codice	Anthropic	140.000+
Feature Dev	Sviluppo feature	Anthropic	131.000+
Playwright	Test browser	Microsoft	118.000+
Ralph Loop	Iterazione autonoma	Anthropic	110.000+
Firecrawl	Web scraping	Firecrawl	Significative
Figma	Design-to-code	Figma	Significative
Sentry	Monitoraggio errori	Sentry	Significative
HuggingFace	ML/AI open source	HuggingFace	Significative
Security Guidance	Sicurezza	Anthropic	25.000+

10.2 Link alle risorse ufficiali

- **Documentazione Claude Desktop:** <https://support.claude.com>
- **Documentazione Claude Code:** <https://code.claude.com/docs>
- **Marketplace plugin:** <https://claude.com/plugins>
- **Repository plugin ufficiali:** <https://github.com/anthropics/claude-plugins-official>
- **Repository plugin knowledge work:** <https://github.com/anthropics/knowledge-work-plugins>
- **Blog Anthropic:** <https://claude.com/blog>
- **Release notes:** <https://support.claude.com/en/articles/12138966-release-notes>
- **Gestione plugin organizzativi:** <https://support.claude.com/en/articles/13837433>
- **Standard Agent Skills:** <https://agentskills.io>

10.3 Glossario dei termini tecnici

Termine	Definizione
Plugin	Pacchetto di funzionalità che estende le capacità di Claude
Skill	File markdown con istruzioni/competenze che Claude carica dinamicamente
Agent (Subagent)	Assistente AI specializzato con contesto e strumenti propri
Hook	Comando/script eseguito automaticamente in momenti specifici del ciclo di vita
MCP (Model Context Protocol)	Protocollo standard per collegare Claude a servizi e strumenti esterni
LSP (Language Server Protocol)	Protocollo per integrare code intelligence in tempo reale
Slash command	Comando invocato con / seguito dal nome (es. /code-review)
Marketplace	Catalogo per la distribuzione e l'installazione di plugin
Frontmatter	Metadati YAML all'inizio di un file markdown (tra --- e ---)
Worktree	Copia isolata del repository Git per operazioni parallele
Compattazione	Riassunto automatico della conversazione per liberare spazio nel contesto
Extended thinking	Modalità di ragionamento esteso disponibile nei modelli più avanzati
Token	Unità di misura del testo processato dai modelli AI
Context window	Quantità massima di testo che il modello può elaborare in una singola interazione